SAFI INSTITUTE OF ADVANCED STUDY(AUTONOMOUS)

Affiliated to University of Calicut, Accredited by NAAC with A++ Grade (3.54)

Recognized by UGC under 2(f), ISO 9001-2015 Certified by AAB

Vazhayoor East, Malappuram Dist, Kerala– 673633

SAFI

Department of Computer Applications

Project Report

Arivolam: All-in-One Educational Platform

Done by,

1. **ABDULLA AHMED - (SIAXBCA015)**
2. **MOHAMMED RABEEH T - (SIAXBCA021)**
3. **MUHAMMED H - (SIAXBCA022)**
4. **MUHAMMED NAVAR M P - (SIAXBCA023)**

Under the Guidance of,

Mrs. Safeena C

2023 - 2026

SAFI INSTITUTE OF ADVANCED STUDY(AUTONOMOUS)

Affiliated to University of Calicut, Accredited by NAAC with A++ Grade (3.54)

Recognized by UGC under 2(f), ISO 9001-2015 Certified by AAB

Vazhayoor East, Malappuram Dist, Kerala– 673633

SAFI

Department of Computer Applications

Project Report

Arivolam: All-in-One Educational Platform

Done by,

1. **ABDULLA AHMED - (SIAXBCA015)**
2. **MOHAMMED RABEEH T - (SIAXBCA021)**
3. **MUHAMMED H - (SIAXBCA022)**
4. **MUHAMMED NAVAR M P - (SIAXBCA023)**

Under the Guidance of,

Mrs. Safeena C

2023 - 2026

SAFI INSTITUTE OF ADVANCED STUDY(AUTONOMOUS)

Affiliated to University of Calicut, Accredited by NAAC with A++ Grade (3.54)

Recognized by UGC under 2(f),ISO 9001-2015 Certified by AAB

Vazhayoor East, Malappuram Dist, Kerala– 673633

SAFI

Department of Computer Applications

CERTIFICATE

Certified that this is the bonafide report on the project work entitled

Done By

1. **ABDULLA AHMED - (SIAXBCA015)**
2. **MOHAMMED RABEEH T - (SIAXBCA021)**
3. **MUHAMMED H - (SIAXBCA022)**
4. **MUHAMMED NAVAR M P - (SIAXBCA023)**

During the academic year 2023-2026 in partial fulfilment of requirement for the award of the degree of,

Bachelor of Computer Applications

Of

University of Calicut, Malappuram Dist., Kerala

Internal Guide **Head of the Department**

Internal Examiner **External Examiner**

SAFI INSTITUTE OF ADVANCED STUDY (AUTONOMOUS)
Arts and Science College | Affiliated to the University of Calicut
Accredited by NAAC with A++ Grade (3.54) | ISO 9001:2015 Certified Institution
+91 483 2880000 mail@sias.edu.in www.sias.edu.in

Date: 11 March 2026

Place: Vazhayur

TO WHOM IT MAY CONCERN

This is to certify that Mr. **ABDULLA AHMED** (REG.NO: SIAXBCA015), Mr. **MOHAMMED RABEEH T** (REG.NO: SIAXBCA021), Mr. **MUHAMMED H** (REG.NO: SIAXBCA022), and Mr. **MUHAMMED NAVAR M P** (REG.NO: SIAXBCA023), 6th-semester students pursuing a Bachelor of Computer Application (BCA) at SAFI Institute of Advanced Study (Autonomous), Vazhayur, Malappuram, have successfully completed their academic project on All-in-One Educational Platform using Next.js, React, and Supabase from December 2025 to March 2026. Throughout the duration of their academic project, they demonstrated punctuality, diligence, and curiosity. We wish them every success in their future endeavors.

PRINCIPAL

ACKNOWLEDGEMENT

First and foremost, we express our deepest gratitude to the Almighty God for His abundant blessings, guidance, and grace that have illuminated our path and granted us strength and resilience throughout our academic pursuits.

We are immensely grateful to the Principal, **Prof. E. P. Imbichikoya** for their visionary leadership, unwavering support, and commitment to fostering an environment of excellence and innovation within our institute.

Our sincere thanks go to the Head of the Department, **Mr. Abdul Samad C** for their invaluable guidance and encouragement, which have been instrumental in nurturing our intellectual curiosity and facilitating our academic growth.

We extend our heartfelt appreciation to our Project Guide **Mrs. Safeena C** for his/her wisdom and guidance have been invaluable assets in our academic journey.

We would also like to express our gratitude to our teachers for their commitment has been instrumental in shaping our understanding and appreciation of the field of Computer Science. To our families, whose unwavering love, encouragement, and sacrifices have been the bedrock of our support system, we offer our deepest appreciation and gratitude.

Lastly, but certainly not least, we extend our heartfelt thanks to our friends, whose companionship, camaraderie, and encouragement have enriched our lives and made this journey memorable and enjoyable. Together, we have achieved significant milestones, and we embark on the next phase of our journey equipped with knowledge, skills, and memories that will last a lifetime.Thank you all for your unwavering support, guidance, and encouragement.

Abdulla Ahmed

Mohammed Rabeeh T

Muhammed H

Muhammed Navar M P

1. ABSTRACT..... 7
2. 1. Problem Statement..... 10
2. 2. Project Motivation..... 10
2. 3. Objectives of the Project..... 10

3. SYSTEM ANALYSIS..... 12
3. 1. Existing System..... 13
3. 2. Proposed System..... 14
3. 4. Feasibility Study..... 14
3. 4. 1. Technical Feasibility..... 14
3. 4. 2. Operational Feasibility..... 14
3. 4. 3. Economic Feasibility..... 15
3. 5. Software Requirement Specifications..... 15
3. 5. 1. Purpose..... 15
3. 5. 2. Scope..... 15
3. 5. 3. Functional Requirements..... 15
3. 5. 4. Non-Functional Requirements:..... 16

4. SYSTEM DESIGN..... 18
4. 1. System Architecture Diagram..... 19
4. 2. Data Flow Diagrams..... 19
4. 2. 1. Level 0..... 19
4. 2. 2. Level 1..... 20
4. 2. 2. 1. Admin..... 20
4. 2. 2. 2. Faculty..... 21
4. 2. 2. 3. Student..... 22
4. 3. ER Diagram..... 23
4. 4. Table Design..... 24
4. 4. 1. attendance..... 24
4. 4. 2. periods..... 24
4. 4. 3. exams..... 26

5. IMPLEMENTATION..... 27
5. 1. Tools, Languages, and Frameworks..... 28
5. 2. Code Explanation..... 28
5. 3. Frontend / Backend / UI Implementation..... 28
5. 4. Model Logic..... 28

6. TESTING AND VALIDATION..... 29
6. 1. Unit Testing..... 30

6. 2. System Testing..... 30
6. 3. Integration Testing.....30

7. CONCLUSION.....31

8. SCOPE FOR FUTURE DEVELOPMENT..... 33
Advanced Navigation & Accessibility.....34
Artificial Intelligence & Automation..... 34
Hardware Integration & IoT.....34
Strategic Outlook.....3

9. APPENDIX..... 35
9. 1. Source Code..... 36
9. 1. 1. Login..... 36
9. 1. 2. Login Form.....37
9. 1. 3. Map Editor.....40
9. 1. 4. Map Viewer..... 43
9. 2. GUI Screenshots.....46
9. 2. 1. Home..... 46
9. 2. 2. Login..... 46
9. 2. 3. Admin Dashboard..... 47
9. 2. 4. Faculty Dashboard.....47
9. 2. 5. Student Dashboard..... 48
9. 2. 6. Map Editor.....48
9. 2. 7. AI Chat Bot..... 49
9. 2. 8. Social Feed..... 49
9. 3. System Minimum Requirements.....50

10. REFERENCES.....51
10. 1. Book References..... 52
10. 2. Website References..... 52

1. ABSTRACT

ABSTRACT

ARIVOLAM: All-in-One Educational Platform

Arivolam is an innovative digital platform designed to solve the pressing issue of digital fragmentation within educational institutions. By consolidating two distinct yet complementary initiatives into a unified application—Ariv Social and Campusolam—this system provides a transformative experience for students, faculty, and administrators. The existing ecosystem in many colleges suffers from scattered digital tools; information is siloed across disparate portals, leading to inefficiencies, miscommunication, and a poor user experience.

Arivolam rectifies these limitations. Ariv Social functions as a vibrant community hub, enabling students and faculty to share updates, engage in discussions, and foster a connected campus culture. Campusolam serves as the operational backbone, offering a highly interactive and detailed campus map for navigation, room search, and infrastructural awareness, along with dedicated features like real-time attendance tracking and academic management.

Developed utilizing a modern, full-stack technology architecture—featuring Next.js, React, Tailwind CSS for a responsive frontend, and Supabase (PostgreSQL) for a scalable and secure backend—Arivolam ensures a reliable, seamless, and high-performance digital environment. This platform effectively minimizes digital friction, bridging the gap between social engagement and academic administration, and paving the way for a more integrated, intelligent, and efficient campus life.

Technologies Used:
Frontend – Next.js, React, Tailwind CSS, Leaflet/Mapbox (for Campus Navigation)
Backend – Next.js API Routes, Supabase
Database – PostgreSQL (via Supabase)

2. INTRODUCTION

INTRODUCTION

2. 1. Problem Statement

Educational institutions frequently adopt multiple, disconnected software solutions to manage different aspects of campus life. Students might use one platform for checking attendance, another for academic announcements, and yet another for navigating the physical campus. This digital fragmentation results in confusion, data silos, and an overall subpar user experience. Stakeholders struggle to keep track of disparate logins and interfaces, reducing engagement and operational efficiency. There is a critical need for a unified platform that consolidates social connectivity, academic management, and spatial navigation.

2. 2. Project Motivation

The motivation behind Arivolam stems from witnessing firsthand the challenges students and faculty face daily due to fragmented information systems. A student trying to locate a specific classroom or a faculty member attempting to track attendance shouldn't navigate through multiple clunky systems. By integrating these essential services into a single, cohesive application—Ariv Social for community building and Campusolam for logistics and administration—we aim to drastically enhance the quality of campus life. Our goal is to create an intuitive, secure, and modern digital environment that empowers all campus stakeholders.

2. 3. Objectives of the Project

To develop a unified digital platform (Arivolam) combining campus social networking (Ariv Social) and operational management (Campusolam).

To implement an interactive campus navigation system featuring deep-linking for rooms, progressive zooming, and multi-floor support.

To provide a robust attendance tracking system with real-time updates for faculty and students.

To create a secure, role-based architecture distinguishing between common users, students, faculty, administrators, and dev-admins.

To ensure a highly responsive and visually appealing user interface utilizing modern web development frameworks.

2. 4. Scope of the Project

The scope of Arivolam encompasses the design, development, and deployment of a full-stack web application. It includes user registration and intricate role-based authentication. Ariv

Social modules will handle posts, announcements, and user interactions. Campusolam modules will manage interactive campus mapping, room search functionality, and attendance management systems. The project will ensure data security through Supabase Row Level Security (RLS) policies and provide administrative dashboards for data management, while intentionally leaving the physical deployment of hardware (like biometric scanners) outside the current scope.

3. SYSTEM ANALYSIS

SYSTEM ANALYSIS

System analysis for Arivolam involves dissecting the current fragmented state of campus digital tools and defining a structurally sound, unified solution. It requires a deep understanding of user roles—how a student interacts with course material versus how an administrator manages campus map data—and ensuring the proposed architecture supports these diverse needs seamlessly.

3. 1. Existing System

In most educational institutions, the digital infrastructure is highly fragmented, relying on multiple disconnected platforms to manage different aspects of campus life. For instance, institutions often use isolated Learning Management Systems (LMS) like Moodle for academic coursework and separate Academic Management Systems like Embase Pro Suite for administrative tracking. The existing system setup has several limitations:

Digital Fragmentation: Students and faculty are forced to juggle multiple applications, platforms, and logins for different tasks, leading to a disjointed and frustrating user experience.

Lack of Community Building: Existing academic platforms are strictly functional and lack native social networking features, making it difficult for students to connect, collaborate, and build a vibrant campus community.

Navigational Inefficiencies: Traditional campus systems rarely offer integrated digital mapping or wayfinding utilities, leaving new students and visitors to rely on static physical maps or disjointed third-party apps to find their way around.

Data Silos: Information is trapped within separate, non-communicating systems (such as attendance in Embase and assignments in Moodle), preventing users from having a unified, centralized view of their complete campus life.

Outdated User Experience: Legacy institutional software often features non-intuitive, outdated interfaces that are not optimized for modern, seamless mobile interaction, ultimately reducing overall student engagement.

3. 2. Proposed System

The proposed Arivolam system is a consolidated web application engineered to replace these disparate tools.

Key features include

1. Unified Authentication: A single secure login for all features.
2. Role-Based Access Control: Distinct interfaces and permissions for students, faculty, and admins.
3. Ariv Social: A feed for campus updates, events, and academic discussions.
4. Campusolam Map Viewer: An advanced, interactive map with a 6-tier progressive zoom, multi-floor switching, and room search capabilities.
5. Attendance Management: Real-time tracking and reporting for classes.
6. Admin Panel: Centralized control for user management, map data editing, and system configuration.

3. 4. Feasibility Study

A feasibility study confirms that Arivolam is practically achievable within the defined constraints and resources.

3. 4. 1. Technical Feasibility

The chosen stack (Next.js, React, Supabase) is highly robust and industry-standard for building complex, scalable web applications.

Map integration using Leaflet and React-Leaflet is well-documented and highly feasible for the required navigation features.

Supabase provides ready-to-use secure authentication and PostgreSQL database management, significantly reducing backend development overhead.

The system will be hosted on scalable cloud platforms (like Vercel and Supabase), ensuring high availability.

3. 4. 2. Operational Feasibility

The user interface is designed with a focus on modern UX/UI principles, ensuring an intuitive experience for both tech-savvy students and administrators.

Consolidating systems reduces the administrative burden of maintaining multiple platforms.

The distinction between Ariv Social and Campusolam within one app provides a clear operational structure for users.

3. 4. 3. Economic Feasibility

Utilizing open-source and generous free-tier services (like Next.js on Vercel and Supabase) minimizes initial setup and hosting costs.

The system eliminates expenses associated with printing physical notices and maintaining multiple software subscriptions.

The primary cost is development time, which is offset by the significant long-term operational efficiencies gained.

3. 5. Software Requirement Specifications

3. 5. 1. Purpose

The purpose of the Arivolam platform is to provide a unified, secure, and user-friendly digital infrastructure for educational institutions. The system aims to resolve digital fragmentation by replacing disjointed, legacy campus applications with a single, modern platform. It integrates community-building features (Ariv Social) with operational and navigational utilities (Campusolam). The system will allow students and faculty to securely log in, connect socially, and navigate campus resources, while enabling administrators to manage users, roles, and institutional data efficiently.

3. 5. 2. Scope

The Arivolam system is designed to consolidate and simplify campus digital services. It ensures a secure and intuitive experience by offering both social and utility features through a robust authenticated environment. The system will handle the following tasks:

✔ User registration and role-based authentication
✔ Personalized user profiles and social networking
✔ Interactive campus mapping and seamless navigation
✔ Centralized access to academic tools and campus operations
✔ Administrator control for managing users, campus data, and announcements

3. 5. 3. Functional Requirements

Functional requirements define the specific behavior and functions of the Arivolam platform:

<u>User Registration and Authentication</u>

Students, faculties, and administrators should be able to securely register and log in with valid credentials.
Users must be authenticated using a robust backend authentication service.
The system should assign and restrict access based on distinct user roles (e.g., Student, Faculty, Admin).

Social and Community Features (Ariv Social)
Users should be able to create, view, and interact with posts within the campus community network.
The system must provide personalized user profiles that securely display relevant academic and social information.

Campus Utilities and Navigation (Campusolam)
Users should be able to access an interactive digital campus map for comprehensive spatial navigation.
The system should provide centralized access to relevant academic resources and operational tools tailored to the user's role.
Administrator Functions:

Admins should be able to monitor and manage user accounts, permissions, and institutional data.
Admins should have the ability to broadcast campus-wide announcements and updates.
Admins should be able to manage and update map locations and operational settings.

3. 5. 4. Non-Functional Requirements:

Non-functional requirements define the quality attributes and performance of the system:

Performance
The system should maintain rapid page load times and highly responsive interactions by leveraging modern rendering techniques (Server-Side Rendering/Static Site Generation).
The centralized database must efficiently handle high volumes of concurrent reads and writes during peak campus hours.

Security
User authentication and database access must be strictly secured using Row Level Security (RLS) policies.

The system must be protected against common web vulnerabilities, including SQL injection and Cross-Site Scripting (XSS).
Unauthorized access to administrative dashboards or restricted role-based routes must be explicitly denied.

Usability
The system must feature an intuitive, modern, and highly responsive user interface.
The application must function seamlessly across diverse platforms, including desktops, tablets, and mobile devices.
Availability:

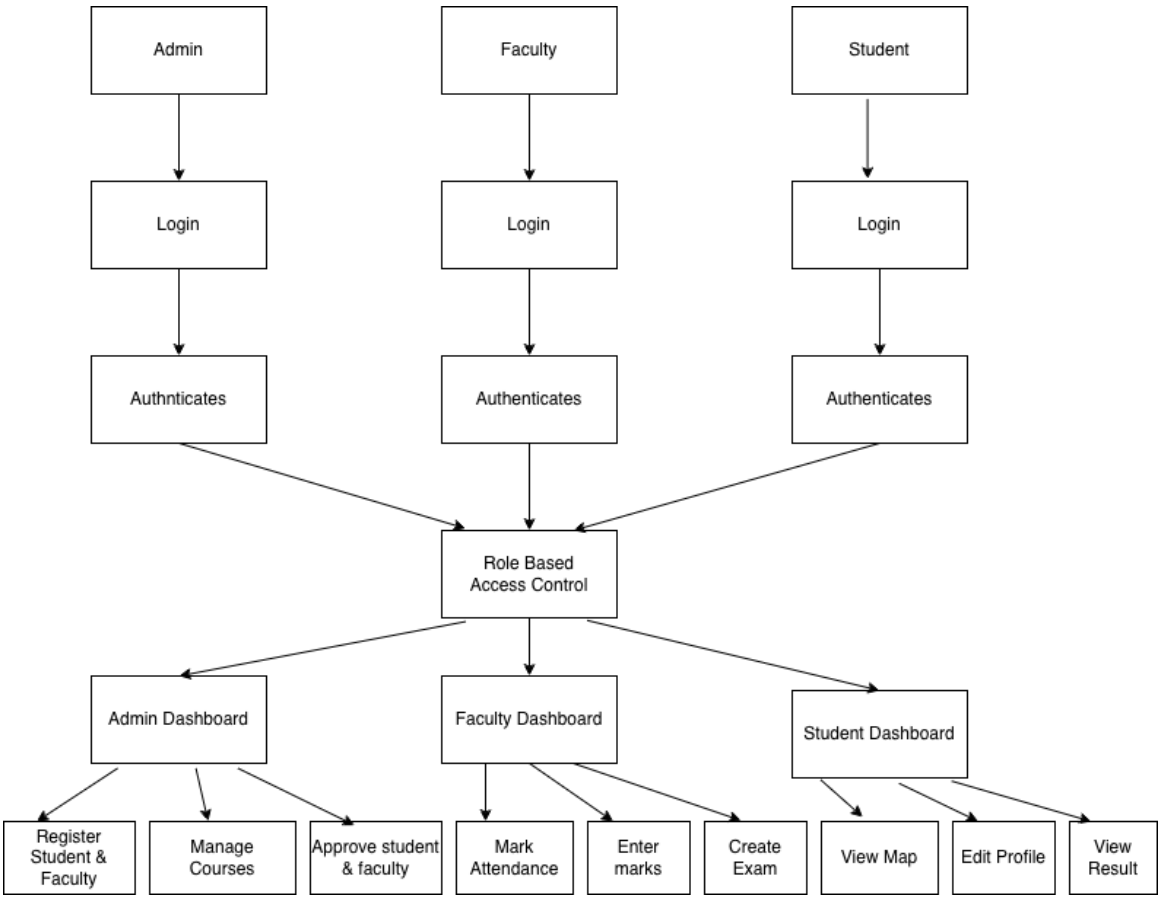
The system should maintain high availability (99.9% uptime) relying on scalable cloud hosting infrastructure.
In the event of a service disruption, the database and application servers must support rapid automated recovery.

Maintainability
The codebase must adhere to modern component-based architectural standards to facilitate easy updates, debugging, and modular maintenance.
The system architecture should be designed with low coupling, ensuring that the Ariv Social and Campusolam modules can manage future scaling or separation efficiently.

4. SYSTEM DESIGN

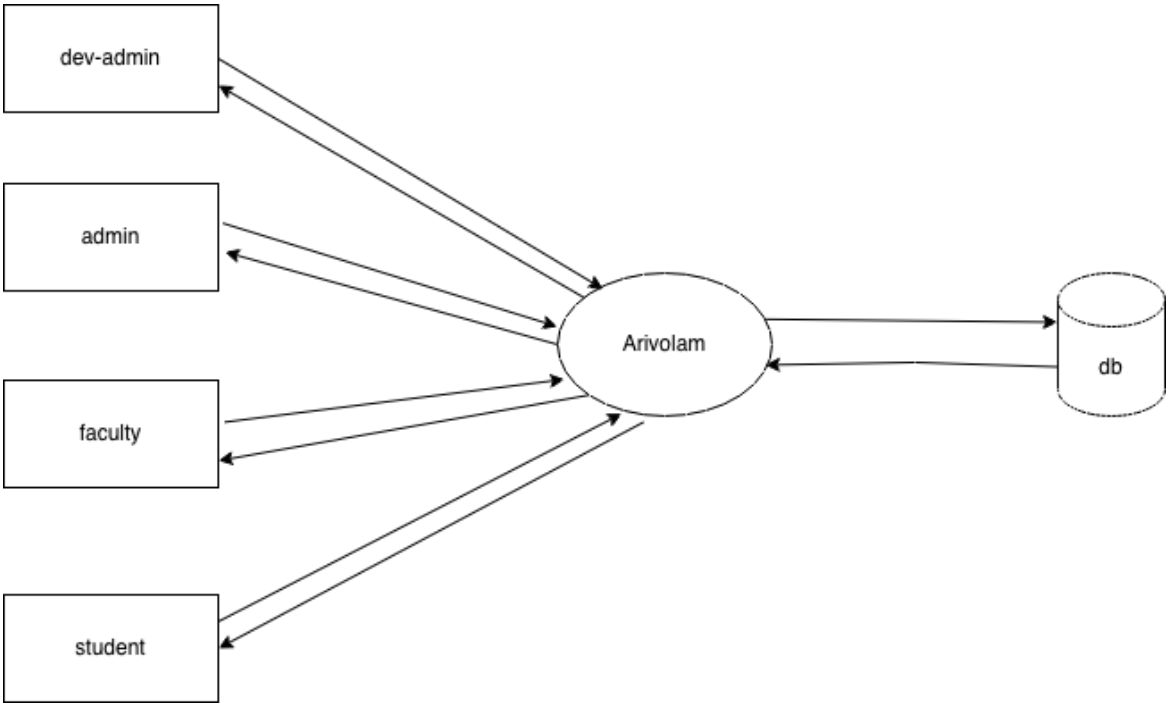
SYSTEM DESIGN

4. 1. System Architecture Diagram



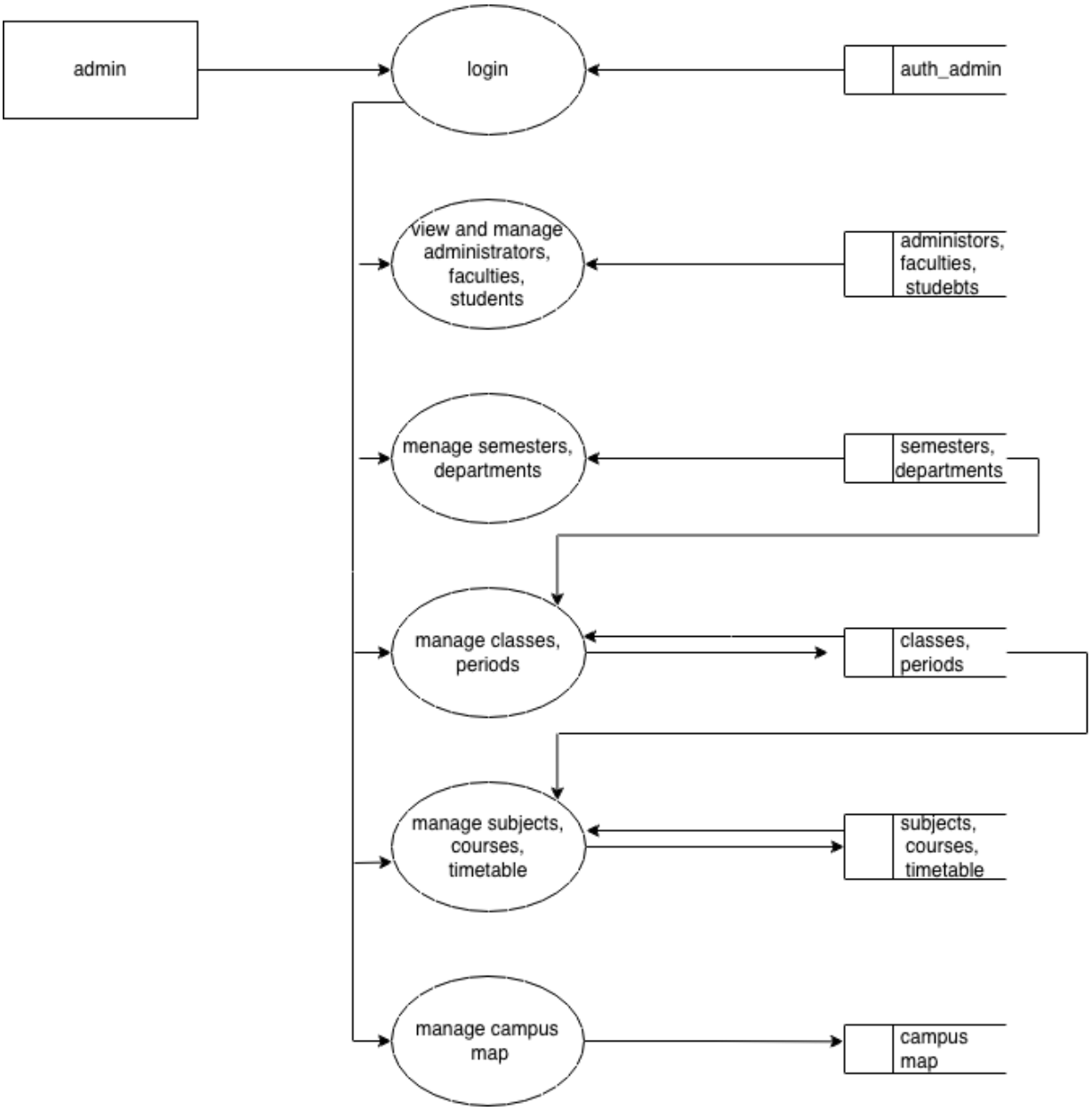
4. 2. Data Flow Diagrams

4. 2. 1. Level 0

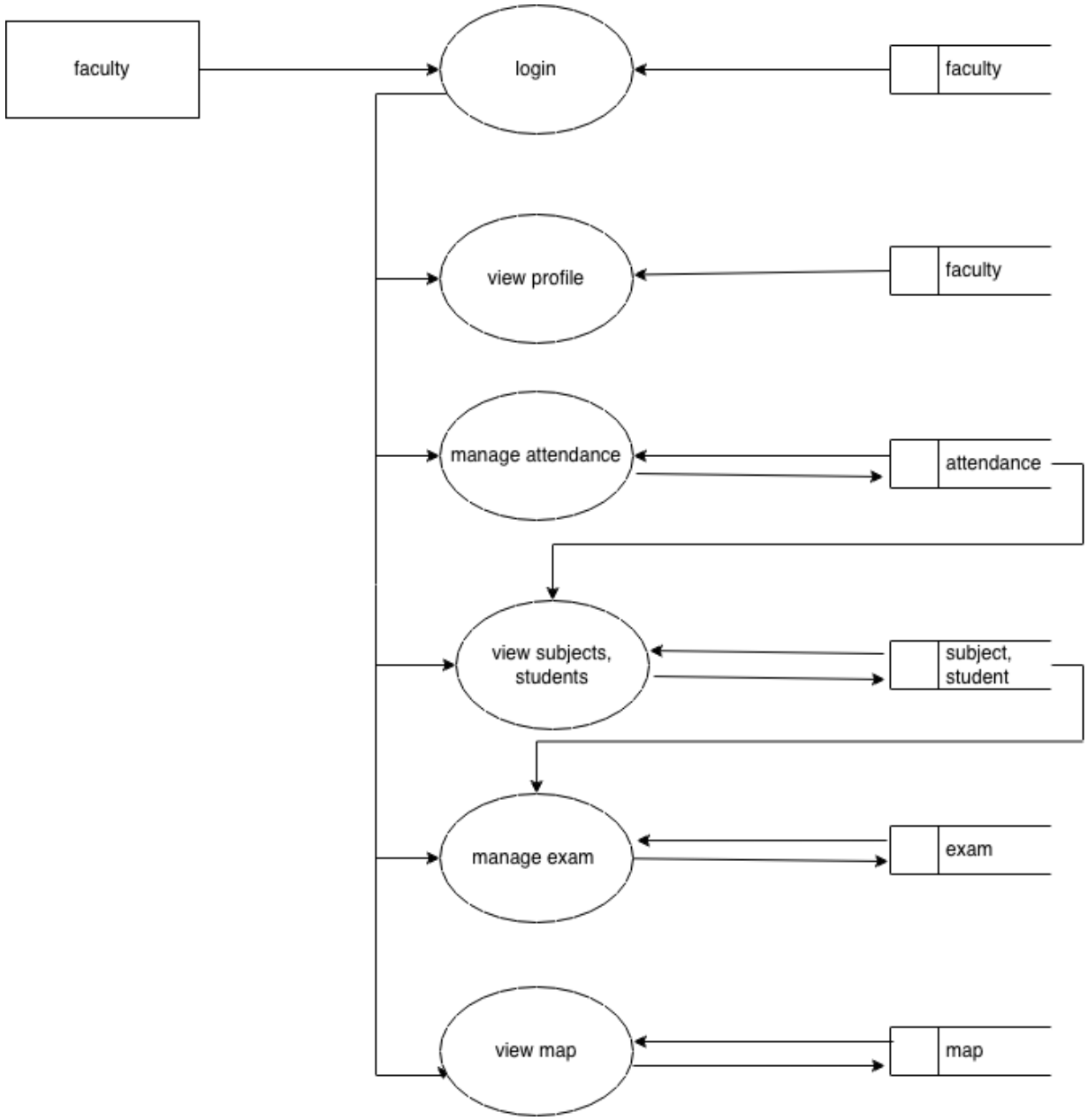


4. 2. 2. Level 1

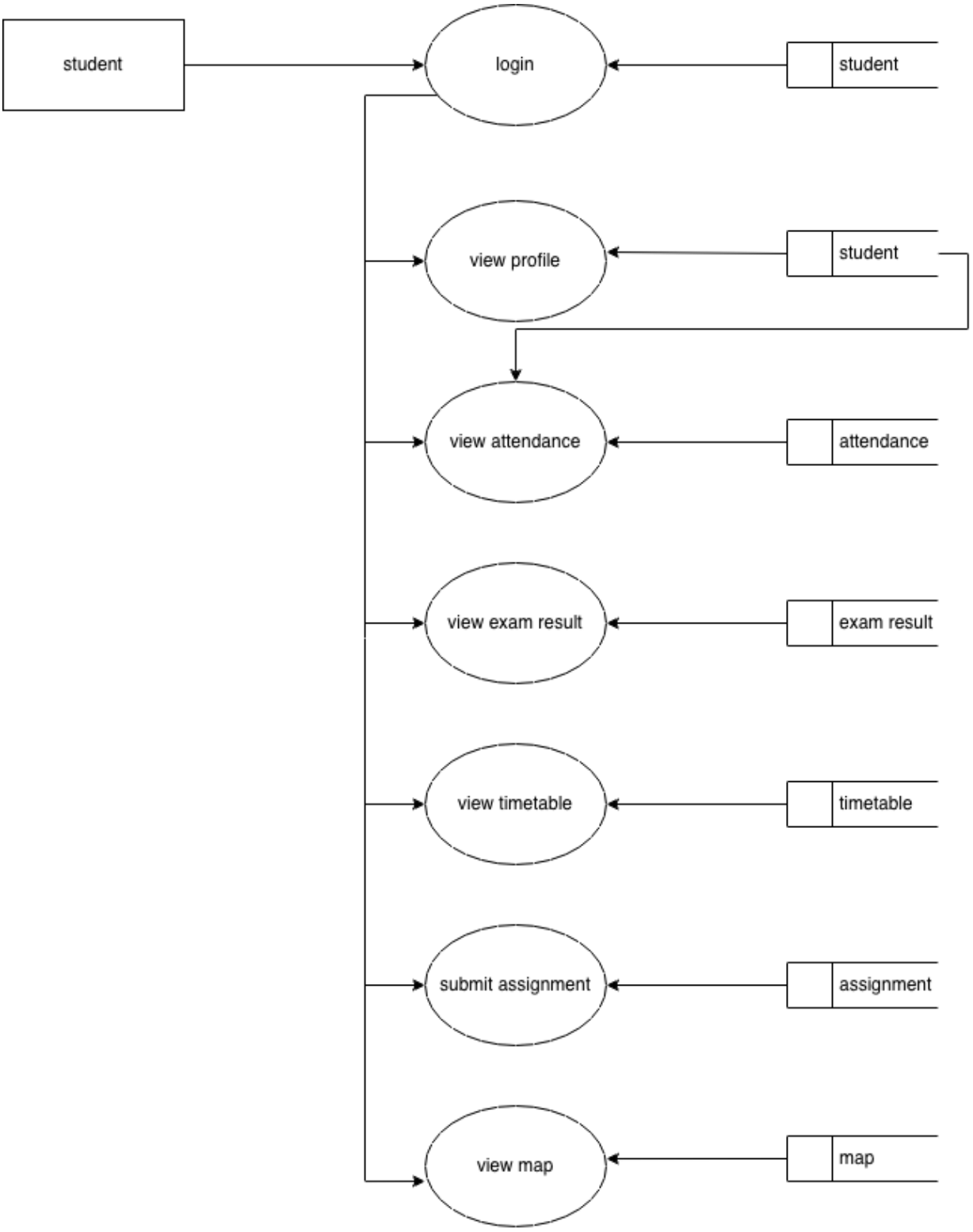
4. 2. 2. 1. Admin



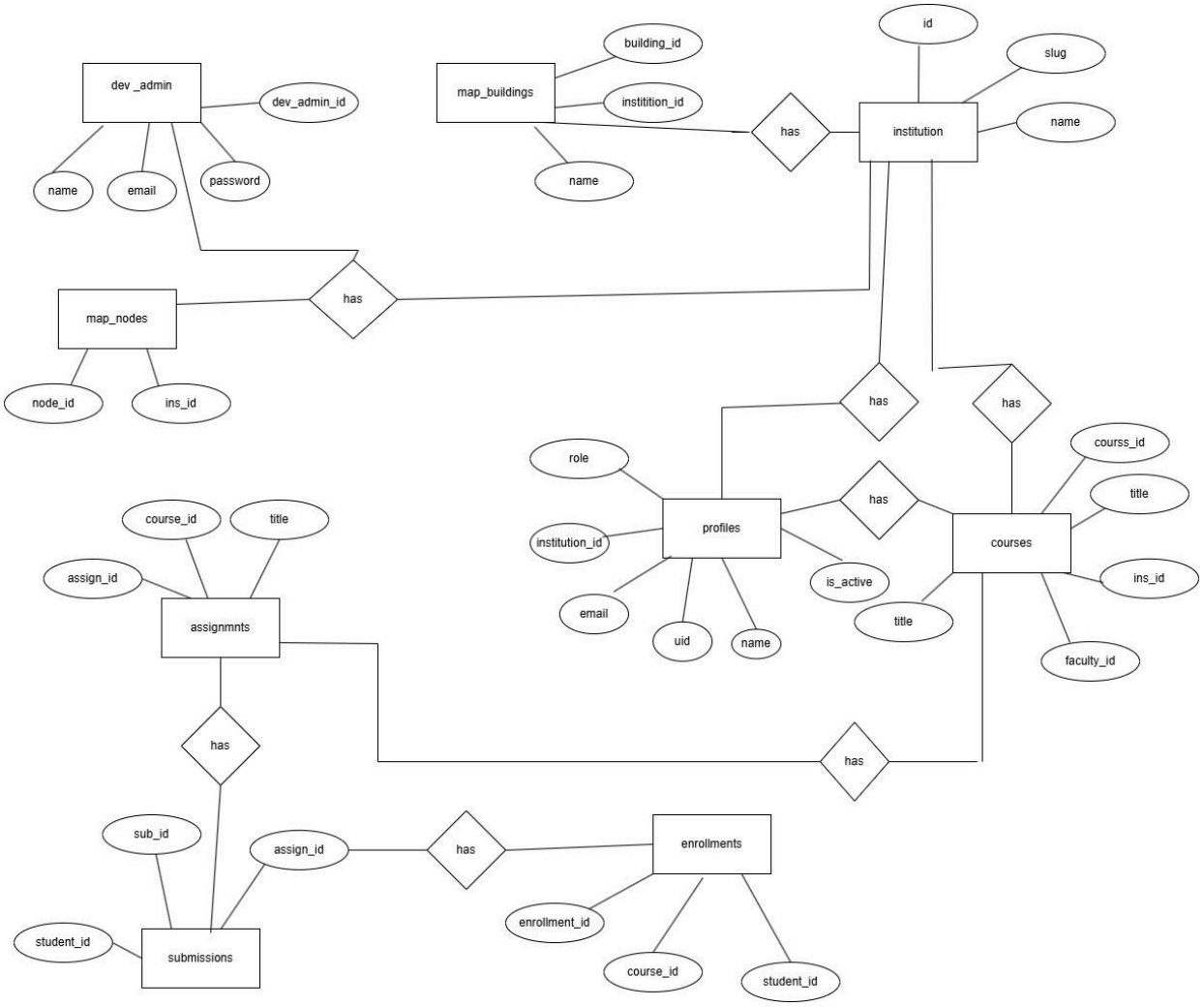
4. 2. 2. 2. Faculty



4. 2. 2. 3. Student



4. 3. ER Diagram



4. 4. Table Design

4. 4. 1. attendance

Purpose: To securely record and track the daily attendance status (Present, Absent, etc.) for each enrolled student per class session.

Primary key: id

column_name	data_type	length_or_precision	description
id	uuid	null	Primary key, Unique identifier for the attendance record.
student_enrollment_id	uuid	null	Foreign key referencing the enrolled student whose attendance is being marked.
timetable_entry_id	uuid	null	Foreign key referencing the specific class session for which attendance is recorded.
date	date	null	The date when the class occurred and attendance was marked.
status	text	null	The student's attendance status (e.g., 'Present', 'Absent', 'Late', 'Unmarked').
marked_by	uuid	null	Foreign key referencing the faculty or admin who recorded the attendance.
remarks	text	null	Optional text for additional notes or reasons regarding the attendance status.
created_at	timestamp with time zone	null	Timestamp recording exactly when the record was created.
updated_at	timestamp with time zone	null	Timestamp recording exactly when the record was last modified.

4. 4. 2. periods

Purpose: To store and sequence the daily bell schedule, defining class periods and breaks for the institution.

Primary key: id

column_name	data_type	length_or_precision	description
id	uuid	null	Primary key, Unique identifier for the schedule/period record.
institution_id	uuid	null	Foreign key referencing the institution this schedule belongs to.
name	text	null	Name or title of the time slot (e.g., "Period 1", "Lunch Break").
start_time	time without time zone	null	The scheduled starting time of the period.
end_time	time without time zone	null	The scheduled ending time of the period.
sort_order	integer	32	Number used to order the periods chronologically for display.
is_break	boolean	null	Flag indicating whether the time slot is a designated break (True/False).
is_active	boolean	null	Status flag indicating if this schedule period is currently active and in use.
created_at	timestamp with time zone	null	Timestamp recording exactly when the record was created.
updated_at	timestamp with time zone	null	Timestamp recording exactly when the record was last modified.

4. 4. 3. exams

Purpose: To define and manage academic examinations, including their scheduling, maximum marks, and weightage toward final grades.

Primary key: id

column_name	data_type	length_or_precision	description
id	uuid	null	Primary key, Unique identifier for the exam/assessment record.
institution_id	uuid	null	Foreign key referencing the institution conducting the exam.
subject_id	uuid	null	Foreign key referencing the specific subject being tested.
class_id	uuid	null	Foreign key referencing the specific class taking the exam.
semester_id	uuid	null	Foreign key referencing the academic semester in which the exam occurs.
exam_type	text	null	Type of the examination (e.g., 'Internal', 'External', 'Midterm', 'Assignment').
name	text	null	Title or name of the exam (e.g., 'First Midterm', 'Final Lab Exam').
max_marks	numeric	6	The maximum possible score achievable for this exam.
weightage	numeric	5	The percentage weight this exam carries towards the final subject grade.
exam_date	date	null	The scheduled date for the examination.
description	text	null	Optional text providing additional details or syllabus coverage for the exam.
is_published	boolean	null	Flag indicating whether the exam details/results are visible to students.
created_at	timestamp with time zone	null	Timestamp recording exactly when the record was created.
updated_at	timestamp with time zone	null	Timestamp recording exactly when the record was last modified.

5. IMPLEMENTATION

IMPLEMENTATION

The implementation phase translates the system design into functional code, building out the Arivolam application using the selected technology stack.

5. 1. Tools, Languages, and Frameworks

Frontend: Next.js (React framework), TypeScript, Tailwind CSS (for styling), GSAP/Framer Motion (for animations).
Map Tools: Leaflet, React-Leaflet, @geoman-io/leaflet-geoman-free (for map editing).
Backend/Database: Supabase (PostgreSQL, Authentication, Realtime datastore).
UI Components: Radix UI, Lucide React (for icons).

5. 2. Code Explanation

The repository is structured following Next.js App Router conventions
/app: Contains all routing, pages, and API endpoints (e.g., /app/profile, /app/map).
/components: Reusable UI elements and complex widgets like the MapViewer or SocialFeed.
/lib and /utils: Helper functions, database clients, and configuration files.

5. 3. Frontend / Backend / UI Implementation

The frontend is built with React components, styled responsively utilizing Tailwind CSS to ensure mobile compatibility. Complex interactive elements, such as the 6-tier progressive zoom map, are implemented using specialized libraries (Leaflet). The backend relies on Supabase, which provides a RESTful API generated instantly from the PostgreSQL schema. We implemented Row Level Security (RLS) policies within Supabase to ensure that, for instance, a student can only view their own attendance, while a faculty member can view attendance for their assigned classes.

5. 4. Model Logic

The core logic resides in how data is structured and related in the PostgreSQL database.
Profiles Table: Stores user metadata, linked via foreign key to the Supabase Auth system. Distinct roles (student, faculty) determine UI rendering.
Map Layout (Buildings, Floors, Rooms): A relational structure where a Room belongs to a Floor, and a Floor belongs to a Building. This allows the progressive map zoom and floor-switcher features to pull data hierarchically.
Attendance Logic: Links a Student to a Course/Session, calculating aggregate presence versus un-marked absences efficiently.

6. TESTING AND VALIDATION

TESTING AND VALIDATION

Testing ensures that Arivolam is reliable, secure, and performs optimally before deployment.

6. 1. Unit Testing

Individual components and utility functions are tested in isolation. For example, testing the logic that structures profile URLs to ensure personal accounts route to /[username] and institutional accounts route to /institution/[username], preventing 404 errors.

6. 2. System Testing

Testing the platform as a complete entity. This involves navigating the application end-to-end: logging in, viewing a social post, switching to the map view, searching for a specific room, and ensuring the UI responds appropriately at every step across different device sizes.

6. 3. Integration Testing

Verifying the interactions between the Next.js frontend and the Supabase backend. Key scenarios include testing image uploads to Supabase storage (bypassing specific restrictive policies safely via backend API routes) and ensuring real-time features like the attendance "Save/Confirm" button correctly update the PostgreSQL database.

7. CONCLUSION

CONCLUSION

In conclusion, the development, testing, and implementation of the Arivolam platform requires a structured and comprehensive approach to ensure its efficiency, security, and usability for all stakeholders, including the Administration, Faculty, Students, and the broader campus community. By systematically following key steps—from robust database design and scalable backend development using Supabase to a highly performant frontend implementation with Next.js and React—the system can be developed with both precision and reliability.

The integration of robust authentication mechanisms ensures secure access control for distinct user roles across both the Ariv Social and Campusolam modules. Comprehensive testing validates the system's functionality and the seamless interaction between its community-building aspects and operational campus utilities. User Acceptance Testing (UAT) plays a crucial role in refining the platform based on feedback from actual end-users, ensuring an intuitive and user-friendly experience that fundamentally resolves the issue of digital fragmentation. Once deployed in a secure and reliable hosting environment, continuous monitoring and maintenance further guarantee the application's stability and high performance during real-world campus activities.

Ultimately, the successful implementation of the Arivolam platform depends on meticulous planning, rigorous testing, and a steadfast commitment to meeting the diverse educational and social needs of its users. By leveraging a modern technology stack to enhance navigational utilities, campus operations, and community engagement, the platform serves as an invaluable tool in bridging the digital campus divide, fostering a deeply connected environment, and significantly enriching the overall educational experience.

8. SCOPE FOR FUTURE DEVELOPMENT

SCOPE FOR FUTURE DEVELOPMENT

The platform holds significant potential for future development, offering opportunities to enhance navigation, automation, and user experience to better serve the evolving needs of educational institutions.

Advanced Navigation & Accessibility

3D & AR/VR Campus Navigation: Upgrading the existing 2D map to incorporate 3D and Augmented/Virtual Reality capabilities. This will provide an immersive, real-time wayfinding experience, allowing users to seamlessly navigate complex campus layouts and locate specific rooms interactively.

Multi-language Support (i18n): Breaking down language barriers by making the platform globally accessible, fostering a more inclusive digital campus experience for a diverse user base.

Artificial Intelligence & Automation

Intelligent AI Agent: Upgrading current features to a fully integrated AI chatbot. This will provide users with proactive, context-aware assistance, handling complex queries regarding campus life, schedules, and administrative processes.

AI Analysis & Auto-Validation: Implementing machine learning algorithms to evaluate students and exams. This will analyze student performance, automate grading/validation processes, and provide predictive insights—reducing the administrative burden on educators while maintaining high academic standards.

Hardware Integration & IoT

RFID Automatic Attendance: Integrating RFID technology to eliminate manual roll calls. This ensures accurate, seamless, and real-time tracking of student presence without disrupting instructional time or requiring manual data entry.

Strategic Outlook

Overall, the future development of the project presents numerous opportunities for innovation and efficiency, driven by advancements in technology and user expectations. By embracing these emerging technologies, enhancing automated protocols, and focusing on user-centric design, the system will continue to evolve and adapt to meet the comprehensive needs of colleges and universities in the digital age.

9. APPENDIX

APPENDIX

9. 1. Source Code

9. 1. 1. Login

```
import { LoginForm } from "@/components/auth/login-form";
import Link from "next/link";
import { Button } from "@/components/ui/button";
import { MoveLeft } from "lucide-react";

export default function LoginPage() {
   return (
            <div className="relative flex min-h-screen flex-col items-center justify-center
overflow-hidden bg-background p-4 md:p-8">
         {/* Background Effects */}
         <div className="absolute inset-0 z-0">
               <div className="absolute left-1/2 top-1/2 h-[500px] w-[500px] -translate-x-1/2
-translate-y-1/2 rounded-full bg-primary/20 opacity-50 blur-[100px]" />
                  <div className="absolute right-0 top-0 h-[300px] w-[300px] bg-blue-500/10
opacity-30 blur-[80px]" />
               <div className="absolute bottom-0 left-0 h-[300px] w-[300px] bg-purple-500/10
opacity-30 blur-[80px]" />
         </div>

         <div className="absolute left-4 top-4 z-20 md:left-8 md:top-8">
            <Link href="/">
               <Button variant="ghost" className="gap-2">
                  <MoveLeft className="h-4 w-4" />
                  Back to Home
               </Button>
            </Link>
         </div>

         <div className="z-10 w-full max-w-md animate-in fade-in zoom-in duration-500">
```

```
            <LoginForm />
          </div>
        </div>
    );
}
```

9. 1. 2. Login Form

```
"use client";

import * as React from "react";
import { motion } from "framer-motion";
import { User, GraduationCap, School, ShieldCheck, ArrowRight } from "lucide-react";
import { Button } from "@/components/ui/button";
import { cn } from "@/lib/utils";
import { login, signup } from "@/app/login/actions"; // Import server actions
import { useTransition } from "react";

const roles = [
        { id: "student", label: "Student", icon: GraduationCap, color: "text-blue-500", bg:
"bg-blue-500/10" },
          { id: "faculty", label: "Faculty", icon: School, color: "text-green-500", bg:
"bg-green-500/10" },
          { id: "admin", label: "Admin", icon: ShieldCheck, color: "text-red-500", bg:
"bg-red-500/10" },
];

export function LoginForm() {
    const [selectedRole, setSelectedRole] = React.useState<string | null>(null);
    const [isPending, startTransition] = useTransition();

    return (
        <div className="w-full max-w-md space-y-8">
          <div className="text-center">
            <h2 className="text-3xl font-bold tracking-tight">Welcome Back</h2>
```

```
        <p className="mt-2 text-sm text-muted-foreground">
          Select your role to access the Arivolam Digital Campus.
        </p>
      </div>

      {/* Note: Roles are currently UI-only. You would likely pass 'role' to the server action
or store it in state to submit.
        For now, we keep the UI but the auth actions are standard email/password.
      */}
      <div className="grid grid-cols-2 gap-4">
        {roles.map((role) => (
          <motion.div
            key={role.id}
            whileHover={{ scale: 1.05 }}
            whileTap={{ scale: 0.95 }}
            onClick={() => setSelectedRole(role.id)}
            className={cn(
              "cursor-pointer rounded-xl border-2 p-4 transition-all duration-200",
              selectedRole === role.id
                ? "border-primary bg-primary/5 ring-2 ring-primary ring-opacity-50"
                : "border-border bg-card hover:border-primary/50"
            )}
          >
                    <div className={`mb-3 flex h-10 w-10 items-center justify-center
rounded-full ${role.bg}`}>
              <role.icon className={`h-6 w-6 ${role.color}`} />
            </div>
            <div className="font-semibold">{role.label}</div>
          </motion.div>
        ))}
      </div>

      <form>
        <div className="space-y-4">
```

```
<div className="relative">
  <input
    name="email"
    type="email"
    placeholder="Email Address"
    required
      className="w-full rounded-md border border-input bg-background/50 px-4 py-3 text-sm ring-offset-background placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:cursor-not-allowed disabled:opacity-50"
  />
</div>
<div className="relative">
  <input
    name="password"
    type="password"
    placeholder="Password"
    required
      className="w-full rounded-md border border-input bg-background/50 px-4 py-3 text-sm ring-offset-background placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:cursor-not-allowed disabled:opacity-50"
  />
</div>

<div className="flex gap-4">
  <Button
    className="w-full"
    size="lg"
    formAction={login}
    disabled={!selectedRole || isPending}
  >
    {isPending ? "Logging in..." : "Login"}
    {!isPending && <ArrowRight className="ml-2 h-4 w-4" />}
```

```
                </Button>
                <Button
                  className="w-full"
                  size="lg"
                  variant="outline"
                  formAction={signup}
                  disabled={!selectedRole || isPending}
                >
                  Sign Up
                </Button>
              </div>
            </div>
          </form>

          <p className="px-8 text-center text-sm text-muted-foreground">
            By clicking login, you agree to our{" "}
            <a href="#" className="underline underline-offset-4 hover:text-primary">
              Terms of Service
            </a>{" "}
            and{" "}
            <a href="#" className="underline underline-offset-4 hover:text-primary">
              Privacy Policy
            </a>
            .
          </p>
        </div>
    );
}
```

9. 1. 3. Map Editor

```
import { createClient } from "@/utils/supabase/server";
import { redirect } from "next/navigation";
import { getSession } from "@/lib/auth";
import { MapEditor } from "@/components/campus/map-editor";
```

```
import { getMapEditorData } from "./actions";

export default async function MapEditorPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;
  const supabase = await createClient();

  // Auth check
  const { data: { user } } = await supabase.auth.getUser();
  const erpSession = await getSession();
  if (!user && !erpSession) redirect("/campus/login");

  // Get institution
  const { data: institution } = await supabase
    .from("institutions")
    .select("id, name, slug")
    .eq("slug", slug)
    .eq("is_active", true)
    .single();

  if (!institution) redirect("/campus");

  // Verify admin role
  let isAdmin = false;
  if (user) {
    const { data: membership } = await supabase
      .from("institution_members")
      .select("role")
      .eq("user_id", user.id)
      .eq("institution_id", institution.id)
      .eq("is_active", true)
      .single();
    if (membership?.role === "admin") isAdmin = true;
```

```
  }
    if (erpSession && erpSession.institution_id === institution.id && erpSession.role ===
"admin") {
      isAdmin = true;
  }

  if (!isAdmin) redirect(`/campus/${slug}`);

  // Fetch all map data
  let mapData;
  try {
    mapData = await getMapEditorData(slug);
  } catch (err: any) {
    return (
      <div className="p-8 text-center text-destructive flex flex-col items-center gap-4">
        <p>Failed to load map data. Please try again.</p>
          <pre className="text-xs bg-muted/50 p-4 rounded-xl text-left text-foreground
font-mono max-w-2xl overflow-auto border">
            {err?.message || String(err)}
        </pre>
      </div>
    );
  }

  // Determine map center deterministically from mapStyle or default
        const mapCenter: [number, number] = mapData.mapStyle?.center_lat &&
mapData.mapStyle?.center_lng
    ? [mapData.mapStyle.center_lat, mapData.mapStyle.center_lng]
    : [11.2274, 75.9104]; // Default: SIAS

  return (
    <div className="p-2 md:p-4">
      <div className="mb-4 flex items-center justify-between">
        <div>
          <h1 className="text-2xl font-bold tracking-tight">Campus Map Editor</h1>
          <p className="text-sm text-muted-foreground">
```

```
                Draw buildings, paths, and navigation routes for {institution.name}
              </p>
            </div>
          </div>
          <MapEditor
            buildings={mapData.buildings}
            pois={mapData.pois}
            navNodes={mapData.navNodes}
            navEdges={mapData.navEdges}
            rooms={mapData.rooms}
            mapCenter={mapCenter}
            slug={slug}
            mapStyle={mapData.mapStyle}
          />
        </div>
    );
}
```

9. 1. 4. Map Viewer

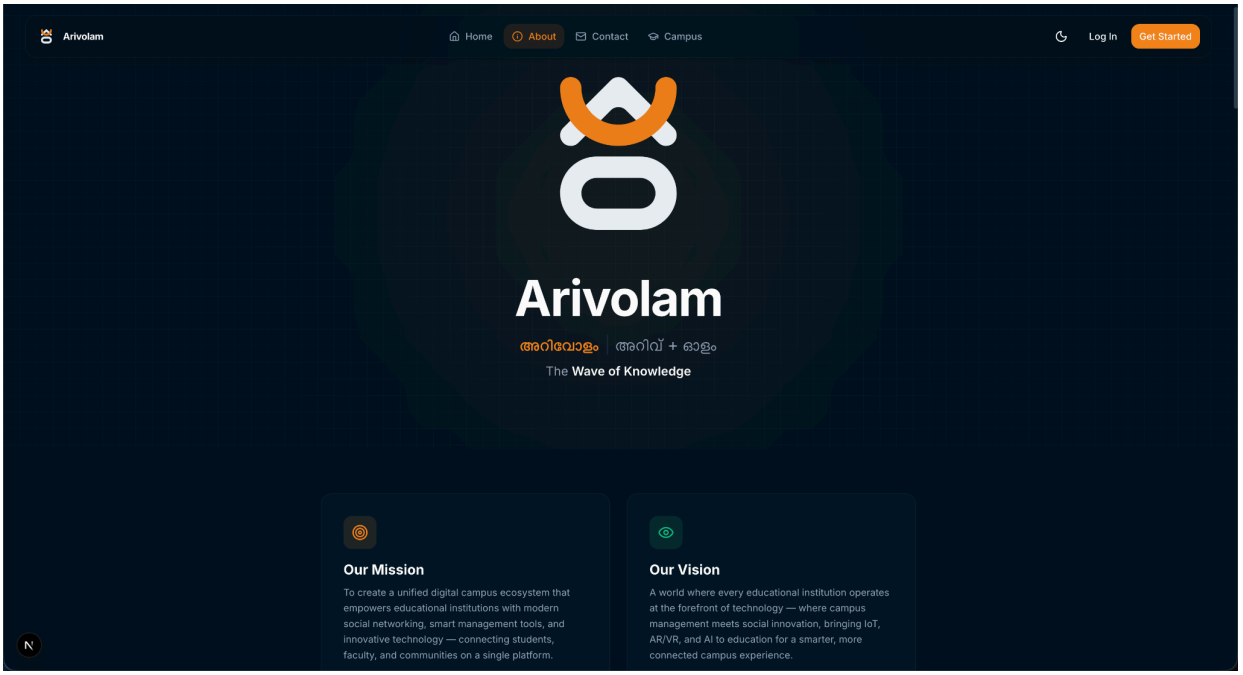
```
import { createClient } from "@/utils/supabase/server";
import { CampusMap } from "@/components/features/campus-map";

export default async function MapPage({
    params,
}: {
    params: Promise<{ slug: string }>;
}) {
    const { slug } = await params;
    const supabase = await createClient();

    // Get institution
    const { data: institution } = await supabase
        .from("institutions")
        .select("id")
        .eq("slug", slug)
        .single();
```

```
if (!institution) {
  return (
    <div className="flex h-[calc(100vh-8rem)] items-center justify-center">
      <p className="text-muted-foreground">Institution not found</p>
    </div>
  );
}

// Fetch buildings with rooms
const { data: buildings } = await supabase
  .from("campus_buildings")
  .select(`
    id, name, short_name, description, category, floors,
    latitude, longitude, geo_polygon, icon, color,
    operating_hours, sort_order, label_visible_zoom, show_polygon,
    canvas_x, canvas_y, canvas_w, canvas_h,
    campus_rooms (
      id, building_id, name, room_number, room_type, capacity,
      description, latitude, longitude
    )
  `)
  .eq("institution_id", institution.id)
  .eq("is_active", true)
  .order("sort_order");

// Fetch POIs
const { data: pois } = await supabase
  .from("campus_pois")
   .select("id, name, category, description, icon, latitude, longitude, building_id, canvas_x,
canvas_y")
  .eq("institution_id", institution.id)
  .eq("is_active", true);

// Fetch navigation graph
const { data: navNodes } = await supabase
```

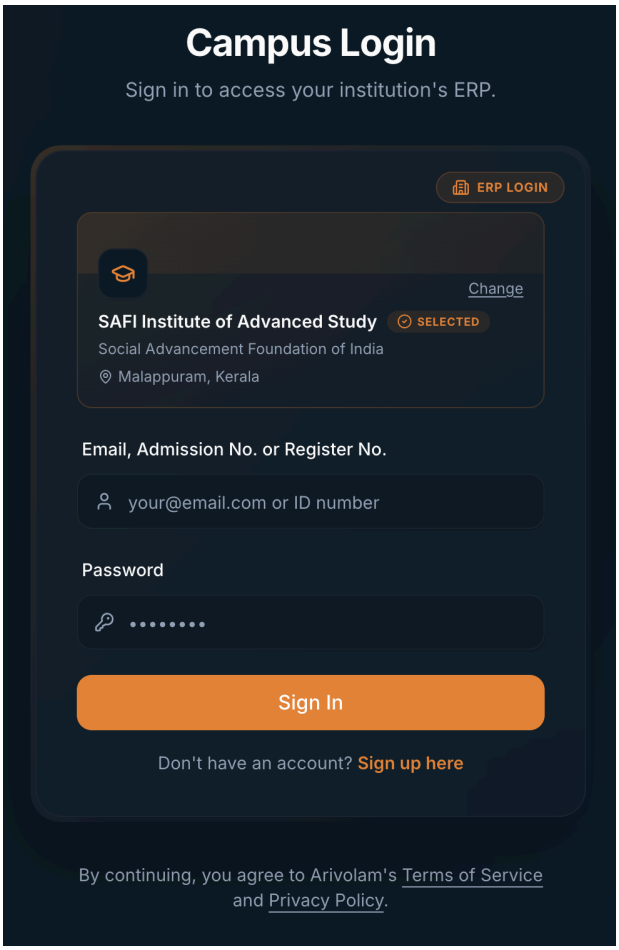
```
    .from("campus_nav_nodes")
    .select("id, latitude, longitude, node_type, label, building_id, canvas_x, canvas_y")
    .eq("institution_id", institution.id);

  const { data: navEdges } = await supabase
    .from("campus_nav_edges")
    .select("id, from_node_id, to_node_id, weight");

  const { data: mapStyle } = await supabase
    .from("campus_map_styles")
    .select("*")
    .eq("institution_id", institution.id)
    .eq("is_active", true)
    .limit(1)
    .maybeSingle();

  // Transform buildings to include rooms as nested property
  const buildingsWithRooms = buildings?.map((b: any) => ({
    ...b,
    rooms: b.campus_rooms || [],
  }));

  return (
    <div className="p-2 md:p-4">
      <CampusMap
        buildings={buildingsWithRooms || undefined}
        pois={pois || undefined}
        navNodes={navNodes || undefined}
        navEdges={navEdges || undefined}
        slug={slug}
        mapStyle={mapStyle || undefined}
      />
    </div>
  );
}
```

9. 2. GUI Screenshots

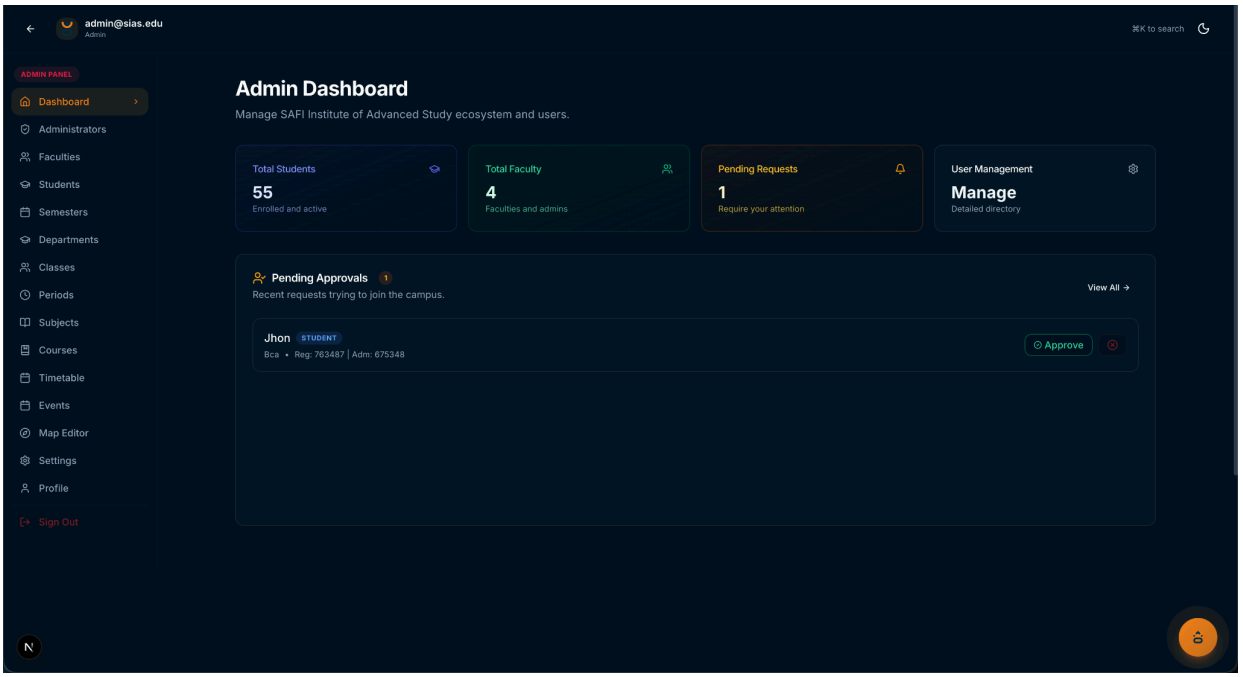
9. 2. 1. Home



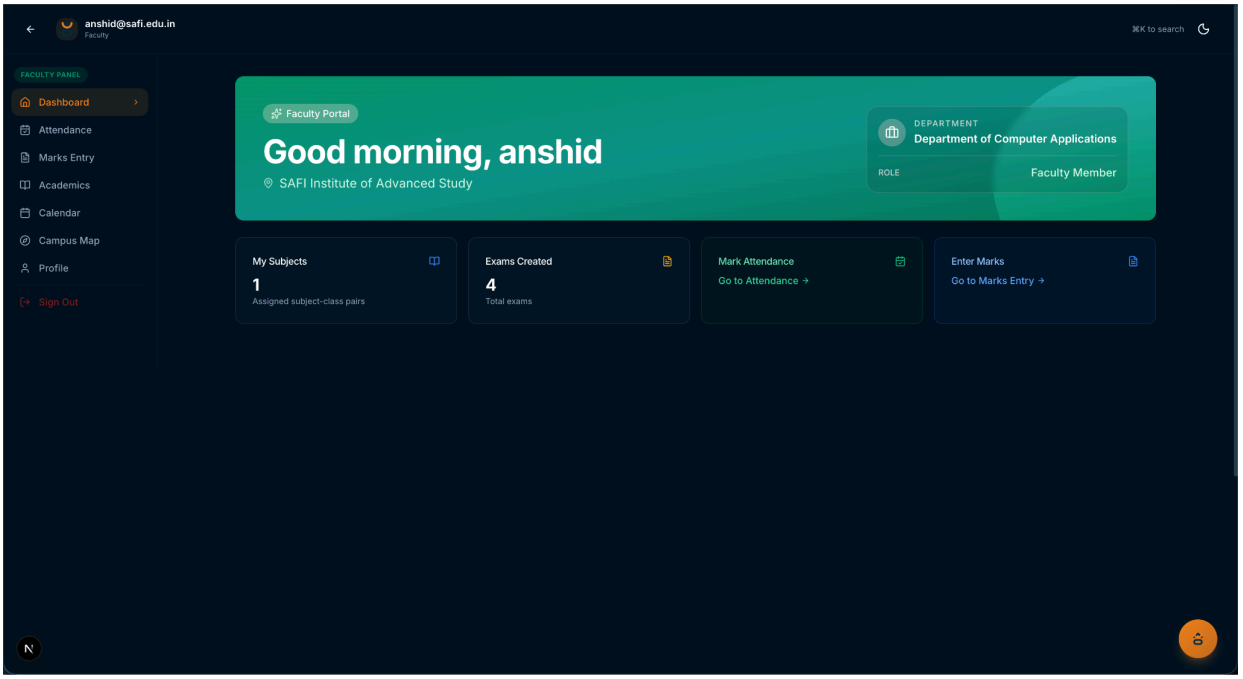
9. 2. 2. Login



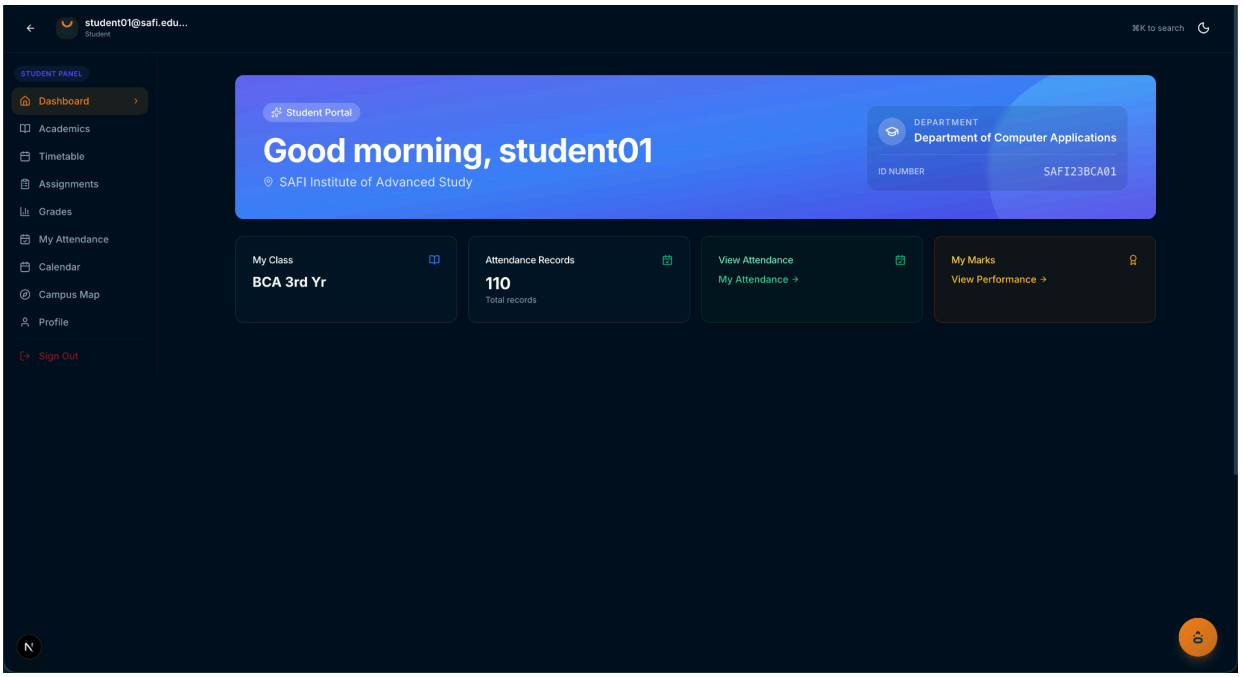
9. 2. 3. Admin Dashboard



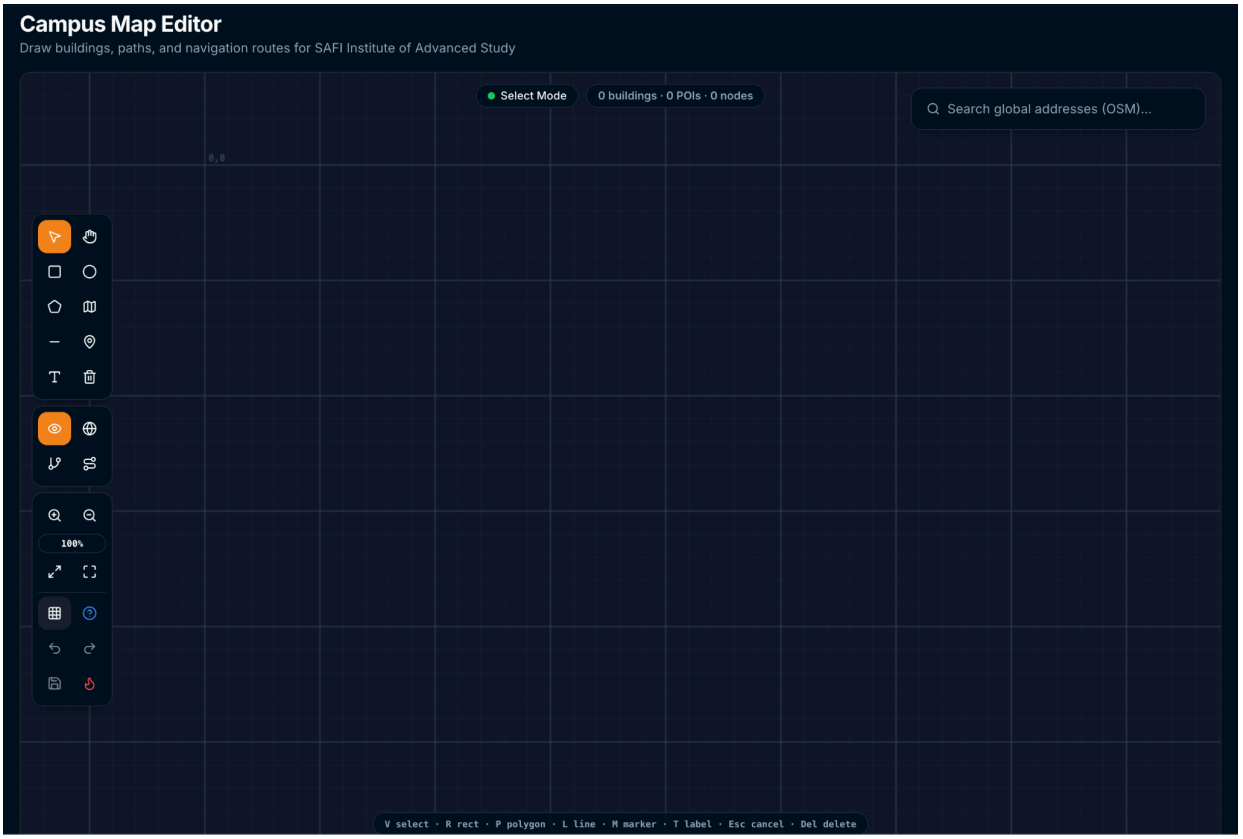
9. 2. 4. Faculty Dashboard



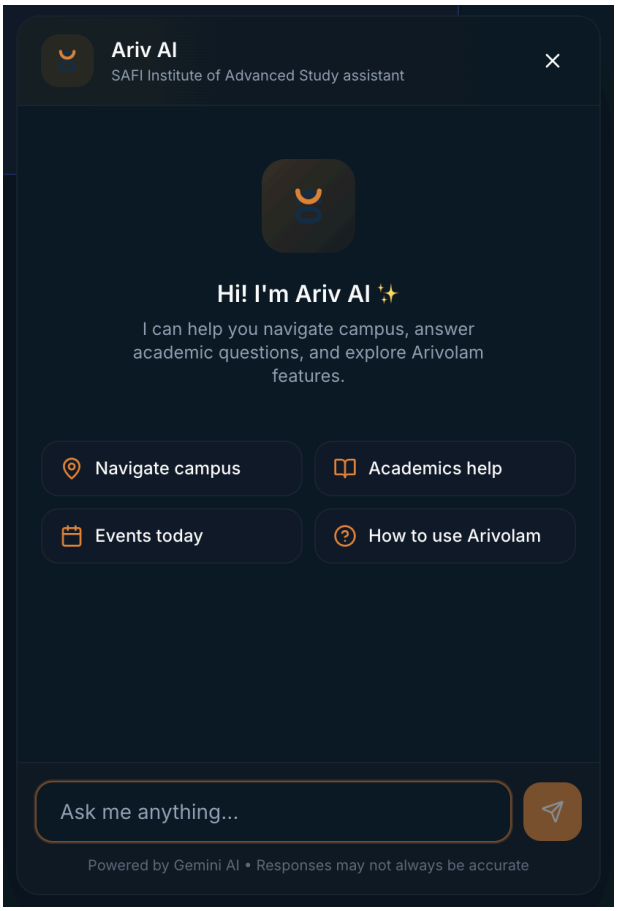
9. 2. 5. Student Dashboard



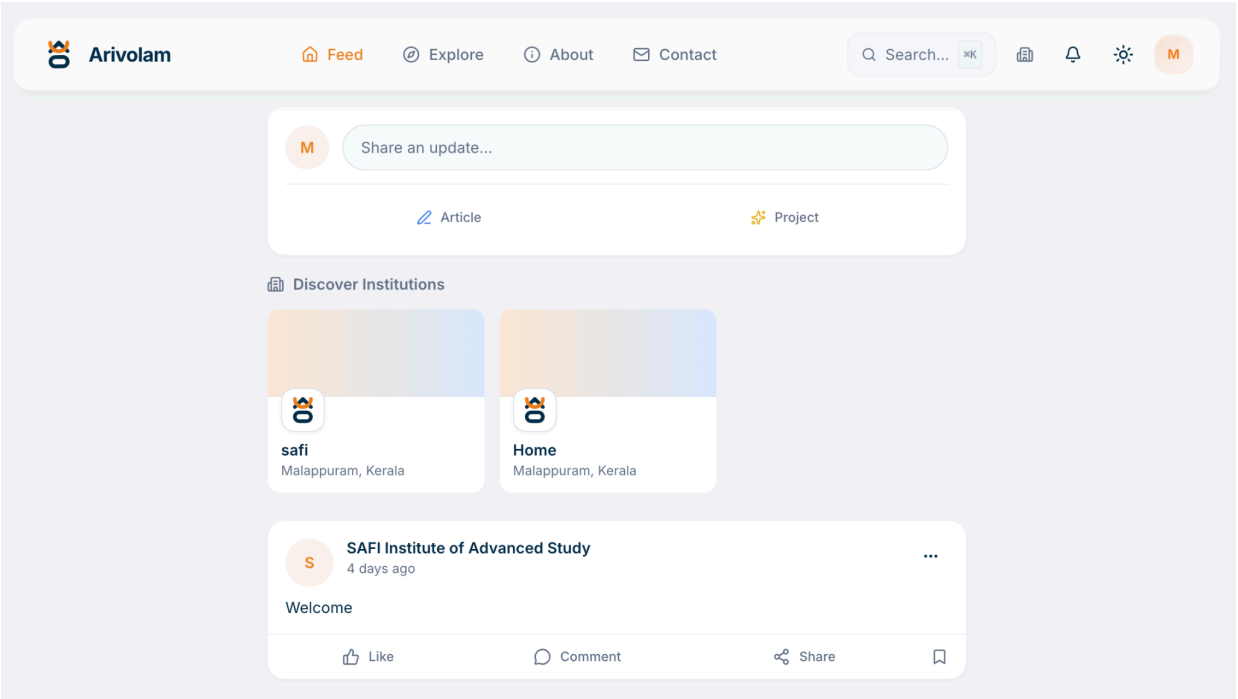
9. 2. 6. Map Editor



9. 2. 7. AI Chat Bot



9. 2. 8. Social Feed



9. 3. System Minimum Requirements

Software configuration:

Operating system: Windows 10/11, macOS, Linux, Android, iOS (via browser)

Frameworks: Next.js 16.x, React 19.x

Styling: Tailwind CSS

Backend/Database: Supabase (PostgreSQL)

IDE: Visual Studio Code

Supported Browsers: Modern versions of Chrome, Safari, Firefox, Edge

Hardware configuration (Client constraints for optimal map rendering):

Processor: Modern Dual-core processor or better

Monitor: Any standard resolution display

RAM: Minimum 4 GB (8 GB recommended for heavy map interaction)

Internet: Stable broadband or 4G connection

10. REFERENCES

10. 1. Book References

1. "**Learning React: Modern Patterns for Developing React Apps**" (2nd Edition) by Alex Banks and Eve Porcello – O'Reilly Media. (Reference for frontend architecture and React hooks).
2. "**Software Engineering**" (10th Edition) by Ian Sommerville – Pearson. (Reference for system analysis, software development life cycle, and testing strategies).
3. "**Database System Concepts**" (7th Edition) by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan – McGraw-Hill. (Reference for database design and normalizations relevant to Supabase).
4. "**Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**" by Martin Kleppmann – O'Reilly Media. (Reference for resolving digital fragmentation and building scalable digital infrastructure).
5. "**Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability**" by Steve Krug – New Riders. (Reference for UI/UX design principles applied to the student and admin modules).

10. 2. Website References

1. **Next.js Official Documentation**
   URL: https://nextjs.org/docs (Reference for routing, server-side rendering, and full-stack React framework concepts).
2. **React Official Documentation**
   URL: https://react.dev/ (Reference for component-based UI design and state management).
3. **Supabase Official Documentation**
   URL: https://supabase.com/docs (Reference for backend-as-a-service, PostgreSQL database management, and authentication).
4. **MDN Web Docs (Mozilla Developer Network)**
   URL: https://developer.mozilla.org/ (Reference for standard web technologies like HTML5, CSS3, and JavaScript best practices).
5. **Vercel Documentation**
   URL: https://vercel.com/docs (Reference for deployment, continuous integration, and application hosting).